

IGWT LECTURES

Sample Thesis

Interactive Graphics and Web Technologies

Combined notes from this folder. Generated from Markdown source.

Interactive Computational Geometry in the Browser

Design and Evaluation of Convex Hull, Delaunay Triangulation, and Bounding-Volume Picking Algorithms for Web-Based Graphics

A sample undergraduate graduation thesis for the program

Interactive Graphics and Web Technologies (IGWT)

Reference document — not for academic submission

This thesis is an educational example for instructors and students. It shows the length, structure, and tone expected of a **20–30 page** project-based undergraduate thesis. Algorithmic claims follow standard literature. Any timing tables are **illustrative templates** and must be replaced with measurements from the author's own implementation. Do not submit this text for credit.

Related notes: Graduation Requirements · 04 Computational Geometry · 03 Reference Thesis

Abstract

Browser-based graphics applications routinely solve geometric problems that are usually taught as separate topics in computational geometry: constructing outlines of point sets, meshing scattered samples, and deciding which object a user has selected. In a web environment these problems must be solved with a small, robust kernel, visualized immediately, and connected to a real-time rendering pipeline.

This thesis presents the design and implementation of an interactive system that exposes three fundamental algorithms in a single JavaScript application: Andrew's monotone-chain convex hull, incremental Delaunay triangulation with edge legalization, and a bounding-volume hierarchy (BVH) for ray-triangle picking. The system is intended both as a teaching tool for an undergraduate computational-geometry course and as a component library for interactive graphics on the web.

The work makes four contributions. First, it specifies a shared geometric kernel based on orientation and incircle predicates, and it documents a consistent policy for degeneracy. Second, it describes a modular architecture that separates predicates, constructions, spatial indexes, and visualization. Third, it implements the three algorithms so that their invariants can be observed step by step. Fourth, it defines a reproducible evaluation protocol that compares each algorithm with a naive baseline in terms of asymptotic cost, measured running time, and qualitative robustness.

The thesis argues that modern web technologies are a practical platform for undergraduate computational geometry, provided that students treat floating-point predicates as a first-class design problem rather than as an implementation detail. The resulting system is suitable as a capstone artifact: it can be demonstrated, tested, documented, and later reused in Three.js scenes for terrain meshing and object picking.

Keywords: computational geometry, convex hull, Delaunay triangulation, bounding volume hierarchy, ray picking, WebGL, JavaScript, interactive graphics, degeneracy, undergraduate thesis

Contents

- 1. 01 Introduction
- 2. 02 Related Work
- 3. 03 Geometric Foundations
- 4. 04 System Design and Algorithms
- 5. 05 Implementation
- 6. 06 Evaluation
- 7. 07 Discussion and Conclusion
- 8. 08 References

Suggested print length: 24–28 pages (about 7,500–8,500 words, plus figures).

Acknowledgements (sample wording)

I thank the instructors of the Interactive Graphics and Web Technologies program for feedback on the course visualizer and on an earlier draft of Chapter 6. Any remaining errors are my own. This sample acknowledgement is a placeholder for a real thesis.

Chapter 1 — Introduction

1.1 Background

Interactive graphics on the web have moved from decorative canvases to complete applications: product configurators, scientific plots, map editors, and educational simulations all run inside a browser. These applications look like rendering problems. Many of their failures, however, are geometric. A click selects the wrong triangle. A concave roof fails to triangulate. A point cloud produces a self-intersecting outline. A terrain mesh contains sliver triangles that shade badly.

Computational geometry studies exactly these questions. Classic textbooks treat convex hulls, triangulations, and spatial search as algorithms on an abstract real RAM [1], [2]. Graphics engines treat the same questions as engine features: collision hulls, navmeshes, and bounding-volume hierarchies [3], [4]. Undergraduate programs in Interactive Graphics and Web Technologies (IGWT) sit between the two traditions. Students must be able to name the predicate that failed, implement the algorithm, and then put the result on a GPU.

The browser is an attractive laboratory for that work. JavaScript and HTML Canvas make visualization cheap. WebGL and libraries such as Three.js make it possible to reuse a 2D kernel inside a 3D scene [5], [6]. The same environment also exposes the difficulty of the subject: IEEE-754 floating-point arithmetic, user-generated degenerate input, and the expectation of interactive frame rates.

1.2 Motivation

Three observations motivate this thesis.

First, students can complete a computer-graphics course without ever writing an orientation test. They call `Raycaster`, `ConvexGeometry`, or a Delaunay package and never see an illegal edge. When a mesh later misbehaves, they have no vocabulary for the failure.

Second, a first course in computational geometry can remain a museum of algorithms unless the implementations are visible. A gift-wrapping animation, a flip of an illegal diagonal, and a BVH that paints pruned boxes teach more than a recurrence on a slide.

Third, the three problems of **outline**, **mesh**, and **pick** appear together in almost every IGWT capstone. A configurator needs picking. A terrain demo needs a triangulation. A 2D editor needs a hull or a simple-polygon test. Implementing them in one system shows that they share a kernel.

1.3 Problem statement

The problem addressed by this thesis is the following.

Design a small, browser-based system that implements convex hull construction, Delaunay triangulation, and BVH ray picking on a shared geometric kernel, so that each algorithm can be visualized, tested on degenerate input, and reused in a web-graphics application.

The problem is not to invent a new hull algorithm. It is to produce a coherent, documented, and evaluable implementation that is honest about floating-point error and suitable as an undergraduate research artifact.

1.4 Research questions

1. How should a JavaScript geometric kernel represent orientation, segment intersection, and the incircle test so that hull, Delaunay, and picking code do not diverge?
2. Which textbook algorithms are appropriate for an interactive undergraduate system with n on the order of 10^2 – 10^4 points or triangles?
3. How can visualization expose the invariant of each algorithm rather than only its final output?
4. What evaluation is sufficient to claim that the implementation is correct and useful, without fabricating experimental results?

1.5 Objectives

1. Specify a kernel of predicates and a degeneracy policy.
2. Implement Andrew's monotone-chain convex hull [7].
3. Implement incremental Delaunay triangulation with legalizing flips [1], [8].
4. Implement a top-down BVH over triangle AABBs and a ray-triangle test [4], [9].
5. Provide a step-mode visualizer for hull and Delaunay, and a prune/visit visualizer for picking.
6. Define an evaluation protocol: functional tests, complexity checks, and timing against naive baselines.

1.6 Scope and limitations

The thesis is restricted to:

- 2D hulls and 2D Delaunay triangulations;
- 3D picking of triangles already in memory (no disk streaming);
- epsilon-based predicates, not exact rational arithmetic;
- n small enough for a teaching visualizer (hull and Delaunay up to a few thousand points; BVH up to a few tens of thousands of triangles).

The thesis does **not** implement Fortune's Voronoi algorithm, Kirkpatrick point location, 3D Delaunay tetrahedralization, or a CGAL-style exact kernel [10], [11]. Those topics are discussed only as related work and future extensions.

1.7 Contributions

1. A compact architecture for teaching computational geometry in the browser.
2. A documented degeneracy policy that is shared by hull, triangulation, and intersection tests.

3. Reference implementations of three algorithms with visible invariants.
4. An evaluation chapter that separates **what must be measured** from **illustrative numbers**, so that a student submitting a real thesis knows what to replace.

1.8 Thesis organization

Chapter 2 reviews algorithms, robustness, and web-graphics practice.

Chapter 3 states the geometric foundations used throughout.

Chapter 4 presents the system design and the three algorithms.

Chapter 5 describes the implementation.

Chapter 6 defines evaluation and reports the form of the results.

Chapter 7 discusses implications, limitations, and future work, then concludes.

Chapter 2 — Related Work

This chapter reviews the algorithms implemented in the thesis, the literature on geometric robustness, and the practice of computational geometry on the web. The aim is not an exhaustive survey. It is to justify the particular choices made in Chapters 4 and 5.

2.1 Convex hulls

The convex hull of a finite point set is a classical primitive. Jarvis’s gift wrapping computes the hull in $\Theta(nh)$ time by walking supporting lines [12]. The algorithm is output-sensitive and excellent when h is tiny; it is quadratic when the points are in convex position. Graham’s scan sorts by polar angle around an extreme point and then walks a stack in linear time, for $O(n \log n)$ total [13]. Andrew’s monotone chain uses the same stack idea but sorts by (x, y) , which avoids polar-angle ties [7]. Chan’s algorithm combines Jarvis and Graham to achieve $O(n \log h)$ [14]. Three-dimensional hulls are substantially harder; randomized incremental construction and the Qhull package are the usual practical references [15], [16].

This thesis uses Andrew’s algorithm as the default implementation and Jarvis as a teaching baseline. The choice follows the undergraduate course: Andrew is easier to implement correctly than Graham, and it is worst-case optimal. Chan’s algorithm is omitted because the extra machinery does not change the visual story for n in the teaching range.

2.2 Triangulations and Delaunay meshes

Every simple polygon with n vertices admits a triangulation with $n - 2$ triangles [2]. Ear clipping is the standard $O(n^2)$ undergraduate method [17]. Faster polygon triangulation proceeds by monotone subdivision [1]. These algorithms fill a *given* boundary. They do not, by themselves, produce well-shaped triangles.

The Delaunay triangulation of a point set is characterized by the empty-circumcircle property: no site lies in the interior of any triangle’s circumcircle [1], [18]. It is the dual of the Voronoi diagram [19]. Incremental insertion with edge flips (or the equivalent Bowyer–Watson cavity) is the usual teaching algorithm [8], [20], [21]. Fortune’s sweep computes the Voronoi diagram, and therefore the Delaunay triangulation, in $O(n \log n)$ time [10]. Constrained Delaunay triangulation respects prescribed edges and is the right tool for navmeshes and GIS polygons [22].

On the web, Delaunator is a widely used 2D implementation [23]. Earcut triangulates polygons, including holes, for map rendering [24]. This thesis reimplements incremental Delaunay rather than calling Delaunator for the main algorithm, because the educational goal is the legalize/flip invariant. A library may be used as an *oracle* in tests.

2.3 Geometric search and picking

Point location asks which face of a subdivision contains a query point. The slab method is simple and uses quadratic space; Kirkpatrick's hierarchy achieves $O(\log n)$ queries with linear space [11]. Range searching on points is solved by kd-trees, range trees, and quadtrees [1], [25].

Interactive graphics almost never uses Kirkpatrick's hierarchy. It uses bounding-volume hierarchies. Clark described hierarchical geometric models for hidden-surface work [26]. Rubin and Whitted and later Kay and Kajiya used nested volumes for ray tracing [27], [28]. Gottschalk, Lin, and Manocha popularized OBB trees for collision [29]. In modern web practice, Three.js performs naive raycasts unless a structure such as `three-mesh-bvh` is added [5], [30]. The Möller–Trumbore test is the usual narrow-phase ray–triangle intersection [9].

This thesis implements a top-down AABB BVH and Möller–Trumbore picking. A 2D kd-tree is discussed as the planar analogue taught in the course, but picking is evaluated in 3D because that is the IGWT use case.

2.4 Robustness

Fortune and Van Wyk, Shewchuk, and the CGAL project documented that naive floating-point predicates fail on near-degenerate input [31], [32], [33]. Shewchuk's adaptive predicates evaluate a floating-point filter and fall back to exact arithmetic only when the error bound requires it [32]. Exact constructions (intersection points, circumcenters) remain difficult even when predicates are exact [34].

Game and web code typically uses an epsilon. That policy is acceptable for teaching and for many interactive applications if it is *consistent* and *tested*. It is not a substitute for an exact kernel in CAD. Chapter 3 states the epsilon policy used here; Chapter 7 discusses when a student project should switch to a library.

2.5 Computational geometry in the browser

JavaScript is an unusual vehicle for computational geometry: it offers only IEEE-754 doubles, no operator overloading, and a single-threaded UI unless workers are used. It also offers Canvas and WebGL, which make algorithm animation straightforward. Several teaching visualizers exist for hulls and Delaunay triangulations; few treat a shared kernel, degeneracy tests, and BVH picking as one thesis-sized system. That combination is the gap this work fills.

2.6 Summary of design decisions

Topic	Adopted approach	Rejected for this thesis
Hull	Andrew, Jarvis as baseline	Chan, 3D Qhull
Mesh	Incremental Delaunay + flips	Fortune, ear clipping as the main mesh
Search	AABB BVH + Möller–Trumbore	Kirkpatrick, full kd-tree picking
Predicates	Shared kernel, epsilon policy	CGAL exact kernel
Platform	JavaScript + Canvas + optional Three.js	Native C++ only

These decisions keep the implementation inside a 20–30 page undergraduate scope while remaining faithful to the literature.

Chapter 3 — Geometric Foundations

This chapter records the definitions and predicates on which the rest of the thesis depends. Implementation details appear in Chapter 5. The notation follows de Berg et al. [1] with a few web-specific conventions.

3.1 Objects

A **point** is a pair (x, y) in $\mathbb{R}^2$, or (x, y, z) in $\mathbb{R}^3$. A **vector** is the difference of two points. A **segment** ab is the set $(1-t)a + t b$ for $t \in [0, 1]$. A **ray** uses $t \geq 0$. A **simple polygon** is a closed chain of vertices whose edges meet only at shared endpoints. A **triangle** is a simple polygon with three vertices. All 2D polygons in this thesis are assumed to be given in counterclockwise (CCW) order unless stated otherwise.

3.2 The orientation predicate

For points a, b, c in the plane define

```
cross(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).
```

This quantity is twice the signed area of triangle abc . The **orientation** is

```
orient(a, b, c) = +1    if cross > ε  
                 = 0    if |cross| ≤ ε  
                 = -1   if cross < -ε.
```

The value $+1$ means that c lies to the left of the directed line ab (CCW). The value -1 means right (CW). Zero is reported as collinear *under the epsilon policy*, not as exact collinearity.

Orientation is translation invariant if it is computed from differences, as above. It is not rotation-invariant under naive scaling: very large coordinates can overflow the meaningful bits of a double [32]. Inputs in this system are expected to lie in a reasonable world box, for example $[-1e6, 1e6]^2$ after a scene transform.

Polar angles are not used as a substitute for orientation. The function `atan2` is slower, wraps at $\pm\pi$, and is a construction, not a predicate.

3.3 On-segment and intersection

Point c lies on segment ab if `orient(a, b, c) = 0` and c lies in the axis-aligned bounding box of ab , expanded by ϵ . Collinearity alone is not sufficient.

Two segments ab and cd are classified as:

- **proper** — they cross at a point interior to both;
- **touch** — they share a point that is an endpoint of at least one segment;

- **overlap** — they are collinear and their intervals overlap in more than a point;
- **none**.

A proper intersection exists when `orient(a, b, c)` and `orient(a, b, d)` have opposite nonzero signs, and the same holds with the roles reversed. The intersection *point* is a construction and is computed only after the predicate says the intersection exists. Parallel segments are detected by a near-zero cross product of their directions; the code must not divide in that case.

3.4 Point in triangle and point in polygon

A point q is strictly inside a CCW triangle abc if `orient(a, b, q)`, `orient(b, c, q)`, and `orient(c, a, q)` are all positive. Boundary cases are reported separately when any orientation is zero and `onSegment` holds.

Point-in-polygon uses the even–odd rule with a half-open edge convention so that a ray through a vertex is not counted twice [2]. The implementation tests the boundary first. This predicate is used by the Delaunay locator and by 2D picking.

3.5 Convexity and hulls

A set S is convex if it contains the segment between any two of its points. The **convex hull** $CH(S)$ is the smallest convex set containing S . For a finite set it is a convex polygon whose vertices are extreme points of S .

Degeneracy policy (hull). Duplicate points, identified within ϵ in both coordinates, are removed before construction. If three hull vertices would be collinear, the middle vertex is dropped. The output is therefore a strictly convex polygon, possibly with fewer vertices than a human drawing of the “visual” hull.

A simple polygon is convex if and only if every consecutive triple has the same nonzero orientation [2]. The bowtie (two segments crossing) shows that the same-turn test is not sufficient without simplicity.

3.6 Incircle and Delaunay legality

Let abc be a CCW triangle and d a fourth point. The **incircle** predicate reports whether d lies inside, on, or outside the unique circle through a , b , and c . In exact arithmetic it is the sign of a 4×4 determinant [32]. This thesis uses that determinant in double precision, with an epsilon on the result, and notes that the test is more fragile than orientation.

An interior edge ad shared by triangles abd and acd is **illegal** if the incircle of one triangle contains the opposite vertex of the other. A **flip** replaces ad by the other diagonal. A triangulation of a point set is Delaunay if and only if every interior edge is legal [1]. Flips terminate because each flip increases the minimum angle.

The empty-circle property of a Voronoi vertex is the same statement in dual form [19]: the circumcenter of a Delaunay triangle is a Voronoi vertex, and its circumcircle contains no site in its interior.

3.7 Bounding volumes and rays

The **axis-aligned bounding box** (AABB) of a set is the product of the intervals of its coordinates. If two AABBs are disjoint, the objects cannot intersect. The converse is false; the AABB is a reject test, not an intersection test.

A **bounding-volume hierarchy** is a binary tree whose leaves hold primitives (here, triangles) and whose internal nodes hold the AABB of their descendants [4], [28]. A query that misses a node box never visits the children.

A ray is $o + t d$ with $t \geq 0$. Intersection with an AABB uses the slab method (per-axis t intervals). Intersection with a triangle uses Möller–Trumbore barycentric coordinates (t, u, v) [9]. A hit requires $t \geq 0, u \geq 0, v \geq 0$, and $u + v \leq 1$.

3.8 Complexity model

Running-time claims use the standard real-RAM / algebraic decision-tree model of computational geometry [1], [2]. They describe the number of predicate evaluations, not JavaScript wall-clock time. Chapter 6 measures wall-clock time separately and treats it as an implementation property.

The relevant bounds for this thesis are:

Problem	Naïve	Implemented
Convex hull	—	Andrew $O(n \log n)$; Jarvis $\Theta(n^2)$
Delaunay, incremental, walk locate	$O(n^2)$ worst case	expected better; lab target $n \leq 2,000$
All-pairs closest / all-pairs intersect	$\Theta(n^2)$	used only as oracles
Ray vs n triangles	$\Theta(n)$	BVH: output-sensitive; worst case still linear

A 2D hull has an $\Omega(n \log n)$ lower bound by reduction from sorting (map x_i to (x_i, x_i^2)) [1]. Andrew is therefore optimal in the worst-case sense used in undergraduate courses.

3.9 Implications for the kernel

All later chapters assume a single module that exports `orient`, `onSegment`, `segmentsIntersect`, `pointInTriangle`, `incircle`, and AABB tests. No algorithm is allowed to reimplement a predicate with a private epsilon. That rule is the main software-engineering contribution of the thesis, and it is the one most often violated in student code.

Chapter 4 — System Design and Algorithms

4.1 Design goals

The system has three design goals, in order of priority.

1. **Correctness of predicates.** A wrong orientation cannot be repaired by a prettier scene graph.
2. **Visibility of invariants.** A student (or examiner) must be able to see the stack, the illegal edge, or the pruned box.
3. **Reuse in graphics.** The same hull, mesh, and BVH modules must be callable from a Canvas demo and from a Three.js scene.

Performance is a goal only after these three. The evaluation in Chapter 6 therefore compares against naive baselines, not against highly tuned C++ libraries.

4.2 Architecture

The application is organized in four layers.

```
UI / visualizer
  ↓
algorithms (hull, delaunay, bvh)
  ↓
kernel (predicates, AABB)
  ↓
math (vectors, matrices)
```

The visualizer may depend on algorithms. Algorithms may depend on the kernel. The kernel may not depend on the visualizer. This rule prevents the common student pattern in which `orient` is copied into a draw function and later edited in only one place.

Figure 4.1 (placeholder). Layer diagram with the four boxes and the allowed arrows.

A scene in the optional Three.js view is an adapter: it converts a Delaunay triangulation into a `BufferGeometry` and a BVH query into a highlight material. It does not own a second kernel.

4.3 Shared data types

Points are plain objects `{x, y}` or `{x, y, z}`. A 2D triangle is three vertex indices into a point array. A half-edge structure is used internally by the Delaunay module so that an edge can find its twin and its two faces [1]. The visualizer never mutates the kernel's arrays; it receives snapshots.

The degeneracy policy of Chapter 3 is implemented once in `kernel/policy.js` as named constants `EPS_ORIENT`, `EPS_SNAP`, and `EPS_INCIRCLE`. Chapter 5 explains how they are chosen.

4.4 Convex hull: Andrew's monotone chain

4.4.1 Algorithm

Given a set S of n points:

1. Remove duplicates within `EPS_SNAP`.
2. Sort the remaining points lexicographically by (x, y) .
3. Build the lower hull left to right. While the last three points of the stack do not make a left turn (`orient  $\leq 0$`), pop.
4. Build the upper hull by scanning the sorted array from right to left with the same rule.
5. Concatenate the two chains, omitting the duplicated endpoints.

4.4.2 Invariant

After each insertion into the lower stack, the stack is a strictly left-turning chain from the leftmost point to the current point, and every processed point lies on or above that chain.

4.4.3 Complexity

Sorting dominates: $O(n \log n)$. Each point is pushed and popped at most once per chain, so the scans are $O(n)$. The algorithm is therefore $O(n \log n)$ and $O(n)$ extra memory.

4.4.4 Jarvis as baseline

Jarvis march is implemented for comparison and for step-mode teaching [12]. It starts at the lowest-then-leftmost point and, at each vertex, scans all points to find the next supporting direction. Its time is $\Theta(n h)$. Chapter 6 uses a circular point set ($h = n$) and a triangular cloud ($h = 3$) to make that bound visible.

4.5 Delaunay triangulation: incremental legalization

4.5.1 Algorithm

The implementation follows the incremental flip algorithm [1], [8].

1. Create a **super-triangle** that contains all input points.
2. For each point p in random order:
 - locate the triangle t that contains p by walking from the previous triangle, using `pointInTriangle`;
 - split t into three triangles (or split an edge if p lies on one);
 - legalize every new interior edge: if an edge is illegal under `incircle`, flip it and legalize the two edges that become exposed.
3. Delete every triangle that shares a vertex with the super-triangle.

4.5.2 Invariant

After each insertion and its legalize cascade, the mesh of inserted points (plus the super-triangle) is Delaunay.

4.5.3 Location and complexity

Walking is expected to be cheap on well-shaped meshes and linear in the worst case. Combined with $O(n)$ work per insertion in the worst case, the implementation is $O(n^2)$ in the worst case. That is acceptable for the teaching range. A student who needs larger n should add a location structure or call Delaunator [23].

4.5.4 Why not ear clipping as the main mesh

Ear clipping triangulates a *polygon* [17]. It is implemented in the course labs and may be used to fill a UI shape. It is the wrong main algorithm for a terrain or a point cloud, because it does not optimize triangle shape. Chapter 6 includes one qualitative figure comparing an ear-clipped concave polygon with a constrained-style mesh of the same vertices; a full constrained Delaunay implementation is left as future work.

4.6 BVH picking

4.6.1 Build

Triangles are placed in an array. A node is built recursively:

- if the bucket has at most L triangles ($L = 4$ in the default), store them as a leaf;
- otherwise split the longest axis of the centroid AABB at the median centroid and build two children;
- store the AABB of all triangles in the node (not only of the centroids).

4.6.2 Query

```
pick(node, ray, best):
    if ray misses node.aabb: return
    if node is leaf:
        for each triangle:
            if Möller-Trumbore hits and  $t < \text{best.t}$ : update best
    else:
        pick(node.left, ray, best)
        pick(node.right, ray, best)
```

The visualizer records visit/prune counts. Closest-hit is required; any-hit would be enough for shadows but not for selection.

4.6.3 Complexity

Build is $O(n \log n)$ with median splits. A query is $O(n)$ in the worst case (a ray that stabs every box) and much smaller on typical scenes with spatial locality [4]. Chapter 6 reports visit counts, which are more informative than a single millisecond number.

4.7 Visualization as part of the design

Each algorithm exposes a **step iterator**.

- Hull: one stack operation per step.
- Delaunay: one insert, or one flip, per step.
- BVH: one node visit per step, with the current box highlighted.

This is not cosmetic. Several bugs in early drafts (a missing twin after a flip; a hull that kept a middle collinear point) were found only when the iterator was drawn. The thesis therefore treats the visualizer as a debugging instrument, not as a user-interface extra.

4.8 Application adapters

Two adapters demonstrate reuse.

Terrain adapter. A height function $z = f(x, y)$ is sampled on a jittered grid. The (x, y) samples are triangulated. The resulting index buffer is uploaded to Three.js. Normals are computed from adjacent faces.

Configurator adapter. A small assembly of boxes and imported glTF triangles is wrapped in the BVH. A click produces a ray in world space; the hit triangle id maps to a part name and a highlight.

Neither adapter is a complete commercial product. They exist to show that the kernel survives contact with a rendering engine.

4.9 Threats the design accepts

The architecture accepts three known weaknesses.

1. Epsilon predicates can be inconsistent on near-zero inputs.
2. Walking point location can be slow or fail if the mesh is temporarily non-Delaunay during a bug.
3. A median-split AABB BVH is not the fastest ray-tracing structure.

These are acceptable in an undergraduate system if they are measured and documented. They would not be acceptable in a CAD kernel.

Chapter 5 — Implementation

5.1 Environment

Item	Choice
Language	JavaScript (ES2020), <code>"type": "module"</code>
2D view	HTML Canvas 2D
3D view	Three.js r160, optional
Tests	Node test runner (or equivalent)
Host	static files; no server-side geometry

The geometric kernel has no DOM dependency and is tested in Node. The visualizer imports the kernel and the algorithms. This split is what makes the test suite possible.

Figure 5.1 (placeholder). Repository tree: `kernel/`, `algo/`, `viz/`, `adapters/`, `tests/`.

5.2 Kernel

The kernel is about 250 lines. The important functions are those of Chapter 3. Two implementation notes matter for reproducibility.

No `atan2` in predicates. Polar sort is never used. Andrew sorts by coordinates. Delaunay uses `pointInTriangle` and `incircle`.

One epsilon file. Changing `EPS_ORIENT` from `1e-9` to `1e-12` is a one-line experiment. Chapter 6 forbids “tuning epsilon until the screenshot looks right” as a way to pass tests. Tests that fail only on pathological coordinates are marked `fragile` and recorded, not deleted.

Intersection points, circumcenters, and barycentric coordinates are isolated in `kernel/construct.js`. A construction is allowed to return `null` when the supporting predicate is zero.

5.3 Convex hull module

Andrew’s algorithm is a direct transcription of Section 4.4. The only non-obvious detail is the treatment of $n \leq 2$ after unique-sort: the function returns the remaining points and does not attempt to build chains.

The step iterator yields `{stack, phase: "lower"|"upper", popped}` so the Canvas view can flash popped vertices. Tests cover:

- empty, one, two points;
- a triangle;
- a square with an interior point;
- an all-collinear set (expected: two endpoints);

- a translated copy of a previous fixture (invariance).

Jarvis is a second file with the same output contract, so the oracle test can compare the two polygons as cyclic sequences.

5.4 Delaunay module

The mesh is a list of triangles and a map from directed edges `(i, j)` to the opposite vertex. A flip of edge `ij` replaces two triangles and updates four map entries. An incorrect update here is the most common implementation bug; the step visualizer draws the twin in red if the map is asymmetric.

Point location starts at the last created triangle and walks across the edge whose orientation says the query is outside. If the walk exceeds a safety bound ($4n$), the implementation falls back to a linear scan and logs a warning. That guard exists because an inconsistent mesh can loop.

The super-triangle is placed using the AABB of the input, expanded by a large margin. After insertion, any triangle incident to a super-vertex is removed. Tests compare the set of edges, for $n \leq 30$, with Delaunator used as an oracle [23]. Edges are compared as undirected pairs. Cocircular inputs are marked `fragile`.

5.5 BVH module

Build is recursive and uses a copy of the triangle-index array. The AABB of a node is the union of triangle AABBs, not the AABB of centroids; using centroids for the stored box is a bug that misses hits.

Möller–Trumbore is implemented from the original paper [9] with a parallel-guard on the determinant. Back-facing triangles are still accepted for picking; a configurator should select a part even if the click hits the underside of a thin board. Shadow rays, if added later, would cull the back face.

The debug query returns `{hit, visits, prunes, boxes[]}` for the visualizer.

5.6 Visualizer

The Canvas application has three tabs: Hull, Delaunay, and Pick.

Common interaction: click to add a point, drag to move, key `N` for a step, key `Space` to run. The Pick tab loads a built-in mesh of a few dozen triangles and a ground plane; a click casts a camera ray (in the 2D teaching view, an orthographic ray).

Color conventions are fixed across the course and the thesis:

Color	Meaning
Blue	current hull stack / current walk triangle
Orange	candidate being tested / visited BVH box
Red	illegal edge / failed ear / back-facing debug
Green	accepted hull edge / legal triangle / hit
Gray	pruned cell or inactive geometry

Figure 5.2 (placeholder). Hull step: lower stack in blue, popped point flashing.

Figure 5.3 (placeholder). Delaunay: circumcircle of the hover triangle, one red illegal edge.

Figure 5.4 (placeholder). BVH: orange visited boxes, gray pruned boxes, green hit triangle.

5.7 Adapters

The terrain adapter samples (x, y) in a unit square, evaluates a sum of two sine waves plus a small jitter, triangulates, and builds a `BufferGeometry`. The jitter avoids a perfectly regular grid, which produces many cocircular quads and stresses `incircle`.

The picking adapter attaches the BVH to a Three.js scene. Three.js `Raycaster` is used only as an oracle in a debug overlay, not as the shipped picker [5].

5.8 Testing discipline

Tests are divided into:

1. **Predicate fixtures** — hand-computed left/right/collinear and the four intersection types.
2. **Oracle tests** — Andrew vs Jarvis; Delaunay vs Delaunator on random $n \leq 30$.
3. **Invariant tests** — after hull, all points have `orient  $\leq 0$` relative to every directed hull edge; after Delaunay, every interior edge is legal.
4. **Degeneracy tests** — duplicates, collinear hull, point on a Delaunay edge, ray through a triangle vertex.

A thesis that reports only screenshots is not considered evaluated. Chapter 6 uses these tests as the functional half of the evaluation.

5.9 What was not implemented

The repository deliberately omits Fortune’s algorithm, hole-aware ear clipping, a persistent DCEL editor, and exact predicates. Those omissions keep the implementation inside the page budget and match the course rule: name them, do not grade a broken Fortune.

Chapter 6 — Evaluation

6.1 Purpose

The evaluation answers three questions.

1. **Functional:** does the implementation satisfy the invariants of Chapters 3 and 4?
2. **Asymptotic:** do the measured growth rates match the textbook bounds on the intended input families?
3. **Practical:** is the system interactive in the teaching range, and does the BVH reduce triangle tests relative to a linear scan?

A fourth question — “is this faster than CGAL / Delaunator / three-mesh-bvh?” — is out of scope. Those libraries are oracles or future replacements, not competitors in an undergraduate thesis.

6.2 Experimental protocol

All timing experiments must be described with the following card. A submitted thesis fills every row with the author’s machine.

Item	Protocol
CPU / RAM	state model and size
Browser or Node	state version
Power	“plugged in, performance mode” on laptops
Input families	listed in Section 6.4
Repeats	warm-up discarded; report median of 11 runs
Clock	<code>performance.now()</code>
What is timed	algorithm only, not Canvas draw

This sample thesis does not report real measurements. The tables below are **templates**. Their numbers are italicized and marked *illustrative*. Replacing them is a requirement for any graded submission.

6.3 Functional results

The test suite of Section 5.8 is the functional evaluation. For a completed project the author should report:

Suite	Cases	Result
Kernel predicates	40	<i>all pass (illustrative)</i>
Hull invariants + oracle	25	<i>all pass (illustrative)</i>
Delaunay legalize + oracle $n \leq 30$	20	<i>18 pass, 2 fragile cocircular (illustrative)</i>
BVH vs linear raycast, 200 random rays	200	<i>same closest triangle (illustrative)</i>

Fragile cases are not failures if they are documented. They *are* failures if they are deleted to make a green badge.

Correctness claims that do not need a clock:

- After Andrew, every input point is on or to the left of every directed hull edge (CCW hull).
- After Delaunay, every interior edge is legal under the same `incircle` used to build the mesh.
- Every BVH hit agrees with a linear Möller–Trumbore scan on the same ray, up to t-ties broken by triangle index.

6.4 Input families

Name	Description	Purpose
Cloud	i.i.d. uniform in the unit square	typical hull / Delaunay
Circle	points on a circle	Jarvis worst case, $h = n$
Hull3	three extreme points + interior cloud	Jarvis best case, $h = 3$
Collinear	points on a line	degeneracy
Grid	regular $m \times m$ grid	cocircular quads
Jitter grid	grid + 1% noise	terrain adapter
Mesh-S	64 triangles (boxes)	BVH debug
Mesh-M	5,000 triangles	BVH timing

6.5 Convex hull

6.5.1 What to measure

Time Andrew and Jarvis on Cloud, Circle, and Hull3 for $n \in \{500, 2,000, 8,000\}$. Record n , h , and milliseconds.

6.5.2 Expected shape (not a substitute for data)

Andrew should be insensitive to h . Jarvis should be clearly slower on Circle than on Hull3 at the same n . If a student's plot does not show that, the implementation or the input generator is wrong.

Table 6.1. Convex-hull running time (*illustrative template*).

n	Family	h	Andrew (ms)	Jarvis (ms)
2,000	Cloud	$O(\log n)$	2.1	3.4
2,000	Circle	2,000	2.0	48
2,000	Hull3	3	2.0	1.1
8,000	Circle	8,000	9.5	720

Figure 6.1 (placeholder). Log-log plot of time vs n for Andrew on Cloud.

6.6 Delaunay triangulation

6.6.1 What to measure

- Build time on Cloud and Jitter grid for $n \in \{200, 800, 2,000\}$.
- Number of flips per insertion (histogram).
- Number of walk steps per insertion.
- Oracle disagreement rate vs Delaunator for $n \leq 30$, 100 random sets.

6.6.2 Qualitative evaluation

Figure 6.2 (placeholder). Same 40 points: a non-Delaunay triangulation with a skinny pair, and the mesh after legalize.

Figure 6.3 (placeholder). Terrain adapter: shaded mesh from the jittered grid.

The author should discuss sliver triangles on Grid (cocircular) versus Jitter grid. If the legalize loop does not terminate, that is a kernel bug, not a “performance issue.”

Table 6.2. Incremental Delaunay (*illustrative template*).

n	Family	Time (ms)	Mean flips / insert	Mean walk steps
800	Cloud	18	3.1	12
2,000	Cloud	95	3.4	22
2,000	Grid	140	unstable / fragile	—

6.7 BVH picking

6.7.1 What to measure

For Mesh-S and Mesh-M, 200 random rays through the scene AABB:

- median triangle tests (linear vs BVH);
- median node visits and prunes;

- median time;
- disagreement count vs linear scan.

6.7.2 Expected shape

On Mesh-M a coherent scene should show an order-of-magnitude drop in triangle tests. A pathological mesh that fills the same AABB (every triangle overlapping the center) will not. The thesis should include one such negative example so that the BVH is not advertised as magic.

Table 6.3. Ray picking (*illustrative template*).

Mesh	Method	Median triangle tests	Median time (ms)	Disagreements
Mesh-M	Linear	5,000	1.8	0
Mesh-M	BVH	40	0.12	0
Overlapping	BVH	2,400	1.1	0

Figure 6.4 (placeholder). One ray, visited boxes in orange, pruned in gray.

6.8 Robustness

The following cases are required screenshots or test logs in a real submission:

1. Hull of an all-collinear set → two endpoints.
2. Duplicate points in the hull input → ignored.
3. Delaunay insertion exactly on an edge → edge split, not a loop.
4. Ray through a triangle vertex → still a unique closest hit (index tie-break).
5. `EPS_ORIENT` increased by 100× → list which tests flip.

Item 5 is the honesty test. If enlarging epsilon “fixes” a random Cloud failure, the algorithm is wrong.

6.9 Usability (lightweight)

Five classmates perform three tasks: add points and compute a hull; insert until they see a flip; pick a part in the configurator adapter. The only recorded metric is task completion (yes/no) and one sentence of confusion. This is not a CHI study. It is enough to catch a visualizer that cannot reset.

6.10 Summary of evaluation claims

What this sample is allowed to claim:

- the *form* of a correct undergraduate evaluation;
- the *invariants* that must hold;
- the *input families* that make complexity visible.

What it is not allowed to claim:

- that the italicized milliseconds were measured;
- that the system is faster than published libraries;
- that epsilon predicates are exact.

A student who copies Table 6.1 into a submitted PDF without running the experiment has fabricated results.

Chapter 7 — Discussion and Conclusion

7.1 Answers to the research questions

RQ1. Kernel.

A single module of orientation, on-segment, intersection, point-in-triangle, incircle, and AABB tests is sufficient for hull, Delaunay, and picking. The important design rule is negative: algorithms must not grow private epsilons. Chapter 5's test split (kernel in Node, visualizer in the browser) is what makes that rule enforceable.

RQ2. Algorithms.

Andrew's chain, incremental legalizing Delaunay, and a median-split AABB BVH are appropriate for n in the teaching range. Jarvis is worth keeping as a baseline because it makes output-sensitive complexity visible. Fortune, Chan, and Kirkpatrick would not change the educational outcome of a 20–30 page thesis.

RQ3. Visualization.

Step iterators found bugs that unit tests missed, especially asymmetric twins after a flip. Visualization is therefore part of the method, not part of the GUI polish. Color conventions shared with the lecture notes reduce cognitive load.

RQ4. Evaluation.

Functional invariants, oracle tests, and timing against naive baselines are enough. Fabricated FPS tables are not. Chapter 6 is intentionally a protocol. A graded thesis must fill it with data.

7.2 Implications for the IGWT program

The system is a bridge between the computational-geometry course and the graphics studio.

- Computer Graphics I can import the Delaunay index buffer instead of a hand-made grid.
- The configurator / interaction course can import the BVH instead of a linear `Raycaster`.
- The capstone can treat the kernel as a dependency, the way a product uses a physics engine, *provided* the student can still point to `orient`.

The larger curricular claim is modest: graduates should be able to look at a graphics bug, name the predicate that failed, and write a test for it. That sentence is also the teaching principle of the course [see 04 Computational Geometry].

7.3 Limitations

1. **Robustness.** Epsilon predicates are inconsistent on some near-degenerate inputs. Shewchuk's adaptive predicates or an exact library would be required for CAD-quality work [32], [33].
2. **Asymptotics.** Incremental Delaunay with walk location is not worst-case optimal. Large GIS point sets need a different locator or a library [23].

3. **3D geometry.** Hulls and Delaunay meshes remain 2D. Picking is 3D, but collision, tetrahedralization, and mesh repair are future work.
4. **Constrained edges.** Rivers, roads, and polygon boundaries are not respected. A CDT is the natural next algorithm [22].
5. **Evaluation in this sample.** Timing tables are templates. The document must not be cited as experimental evidence.

7.4 Future work

1. Replace `incircle` with an adaptive exact predicate and re-run the fragile Grid family.
2. Implement a constrained Delaunay triangulation and compare it with ear clipping on building footprints.
3. Add a SAH (surface-area heuristic) BVH split and compare visit counts with the median split [4].
4. Port the kernel to TypeScript and to a WebGPU compute path for discrete Voronoi (course Week 10).
5. Use the visualizer as required lab software in the computational-geometry course and collect which degenerate cases students still miss.

7.5 Ethical and academic notes

A project-based thesis in this program is a piece of engineering scholarship. It is legitimate to use textbooks, Stack Overflow, and large language models as *tools*. It is not legitimate to submit generated text or generated “measurements” as if they were the student’s experimental work. The present document is a reference for structure and tone. A submitted thesis must describe a system the author actually built.

7.6 Conclusion

This thesis specified a browser-based system for three geometric problems that appear throughout interactive graphics: constructing a convex hull, meshing a point set with a Delaunay triangulation, and picking a triangle with a bounding-volume hierarchy. The algorithms are standard. The contribution is their integration on a shared kernel, their visible invariants, and an evaluation protocol that an undergraduate can actually carry out.

The work supports a simple claim. The web is a sufficient platform for teaching and applying computational geometry at the level required by an Interactive Graphics and Web Technologies program, if — and only if — predicates, degeneracy, and tests are treated as the core of the system rather than as chores to be finished after the shaders look good.

Appendix A — Suggested figure list for the printed PDF

Figure	Caption
1.1	Three failure pictures: wrong pick, sliver terrain, self-intersecting outline
4.1	Four-layer architecture
4.2	Andrew lower/upper stacks on a 12-point set
4.3	One illegal edge and its flip, with circumcircles
4.4	BVH split of a 16-triangle mesh
5.2–5.4	Visualizer screenshots (hull, Delaunay, pick)
6.1	Hull timing plot
6.2	Before/after legalize
6.3	Terrain adapter
6.4	Ray and BVH boxes

Appendix B — Oral defense (sample questions)

These match the program's oral-defense requirement Graduation Requirements.

1. Why is `atan2` the wrong primitive for "left of a line"?
2. Show the input on which Jarvis is quadratic and Andrew is not.
3. Prove, in four sentences, that a Delaunay triangulation has empty circumcircles if and only if every edge is legal.
4. Your `incircle` failed on a regular grid. What did you do, and why is deleting the test unacceptable?
5. A ray hits two triangles at the same t . How do you pick a part?
6. What would you change first if this kernel were used in a medical-measurement tool rather than a teaching demo?

References

IEEE-style numbering, in order of first citation. URLs are optional in a printed copy. Students should replace this list with the editions they actually used and add page-level citations in the text.

- [1] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin, Germany: Springer, 2008.
- [2] J. O'Rourke, *Computational Geometry in C*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 1998.
- [3] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*, 4th ed. Boca Raton, FL, USA: A K Peters / CRC Press, 2018.
- [4] C. Ericson, *Real-Time Collision Detection*. San Francisco, CA, USA: Morgan Kaufmann, 2005.
- [5] R. Cabello et al., "three.js — JavaScript 3D library," [Online]. Available: <https://threejs.org/>
- [6] Khronos Group, "WebGL Specification," [Online]. Available: <https://www.khronos.org/webgl/>
- [7] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Inf. Process. Lett.*, vol. 9, no. 5, pp. 216–219, 1979.
- [8] L. J. Guibas and J. Stolfi, "Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams," *ACM Trans. Graph.*, vol. 4, no. 2, pp. 74–123, 1985.
- [9] T. Möller and B. Trumbore, "Fast, minimum storage ray/triangle intersection," *J. Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997.
- [10] S. Fortune, "A sweepline algorithm for Voronoi diagrams," *Algorithmica*, vol. 2, pp. 153–174, 1987.
- [11] D. G. Kirkpatrick, "Optimal search in planar subdivisions," *SIAM J. Comput.*, vol. 12, no. 1, pp. 28–35, 1983.
- [12] R. A. Jarvis, "On the identification of the convex hull of a finite set of points in the plane," *Inf. Process. Lett.*, vol. 2, no. 1, pp. 18–21, 1973.
- [13] R. L. Graham, "An efficient algorithm for determining the convex hull of a finite planar set," *Inf. Process. Lett.*, vol. 1, no. 4, pp. 132–133, 1972.
- [14] T. M. Chan, "Optimal output-sensitive convex hull algorithms in two and three dimensions," *Discrete Comput. Geom.*, vol. 16, pp. 361–368, 1996.
- [15] K. L. Clarkson and P. W. Shor, "Applications of random sampling in computational geometry, II," *Discrete Comput. Geom.*, vol. 4, pp. 387–421, 1989.
- [16] C. B. Barber, D. P. Dobkin, and H. Huhdanpaa, "The Quickhull algorithm for convex hulls," *ACM Trans. Math. Softw.*, vol. 22, no. 4, pp. 469–483, 1996.
- [17] G. H. Meisters, "Polygons have ears," *Amer. Math. Monthly*, vol. 82, no. 6, pp. 648–651, 1975.
- [18] B. Delaunay, "Sur la sphère vide," *Izv. Akad. Nauk SSSR, Otd. Mat. Estestv. Nauk*, vol. 7, pp. 793–800, 1934.

- [19] F. Aurenhammer, "Voronoi diagrams—A survey of a fundamental geometric data structure," *ACM Comput. Surv.*, vol. 23, no. 3, pp. 345–405, 1991.
- [20] A. Bowyer, "Computing Dirichlet tessellations," *Comput. J.*, vol. 24, no. 2, pp. 162–166, 1981.
- [21] D. F. Watson, "Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes," *Comput. J.*, vol. 24, no. 2, pp. 167–172, 1981.
- [22] L. P. Chew, "Constrained Delaunay triangulations," *Algorithmica*, vol. 4, pp. 97–108, 1989.
- [23] V. Agafonkin, "Delaunator: An incredibly fast JavaScript library for Delaunay triangulation," [Online]. Available: <https://github.com/mapbox/delaunator>
- [24] Mapbox, "earcut: The fastest and smallest JavaScript polygon triangulation library," [Online]. Available: <https://github.com/mapbox/earcut>
- [25] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Commun. ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [26] J. H. Clark, "Hierarchical geometric models for visible surface algorithms," *Commun. ACM*, vol. 19, no. 10, pp. 547–554, 1976.
- [27] S. M. Rubin and T. Whitted, "A 3-dimensional representation for fast rendering of complex scenes," *Comput. Graph. (Proc. SIGGRAPH)*, vol. 14, no. 3, pp. 110–116, 1980.
- [28] T. L. Kay and J. T. Kajiya, "Ray tracing complex scenes," *Comput. Graph. (Proc. SIGGRAPH)*, vol. 20, no. 4, pp. 269–278, 1986.
- [29] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *Proc. SIGGRAPH*, 1996, pp. 171–180.
- [30] G. Johnson, "three-mesh-bvh," [Online]. Available: <https://github.com/gkjohnson/three-mesh-bvh>
- [31] S. Fortune and C. J. Van Wyk, "Efficient exact arithmetic for computational geometry," in *Proc. 9th Annu. Symp. Comput. Geom.*, 1993, pp. 163–172.
- [32] J. R. Shewchuk, "Adaptive precision floating-point arithmetic and fast robust geometric predicates," *Discrete Comput. Geom.*, vol. 18, pp. 305–363, 1997.
- [33] CGAL Project, *CGAL User and Reference Manual*. [Online]. Available: <https://www.cgal.org/>
- [34] L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, and C. Yap, "Classroom examples of robustness problems in geometric computations," *Comput. Geom. Theory Appl.*, vol. 40, no. 1, pp. 61–78, 2008.
- [35] D. M. Mount, "CMSC 754: Computational Geometry lecture notes," Univ. of Maryland. [Online]. Available: <https://www.cs.umd.edu/~mount/754/>
-

How to cite this sample (for instructors)

Do not cite the italicized tables as data. Cite the document, if at all, as curriculum material:

IGWT Program, "Interactive Computational Geometry in the Browser" (sample undergraduate thesis, 20–30 pages), Lecture notes, 2026.